



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Class Activity – 15/07/2026

Jana Malakondaiah

192424223

1. Smart City Air Quality Monitoring: A city has installed 150 air quality sensors across different locations. The sensors record PM2.5, NO2, CO, temperature, and humidity every hour. Decision-makers struggle to identify pollution hotspots because of overlapping values.

Code:

R Program	Python Code
<pre>install.packages("ggplot2") library(ggplot2) air<-data.frame(Location=c("L1","L2","L3","L4","L5","L6","L7","L8"), NO2=c(20,55,30,75,25,60,15,70), CO=c(0.5,1.8,1.2,2.5,0.8,1.6,0.4,2.2), Temp=c(30,34,31,36,29,33,28,35), Humidity=c(65,55,60,45,70,58,75,50)) print(air) # Sequential Palette ggplot(air,aes(x=Temp,y=PM25,color=PM25))+ geom_point(size=5)+ scale_color_gradient(low="lightgreen",high="darkgreen")+</pre>	<pre>import pandas as pd import matplotlib.pyplot as plt df=pd.DataFrame({"Location":["L1","L2","L3","L4","L5","L6","L7","L8"], "PM25":[35,90,60,120,45,80,25,110],"NO2":[20,55,30,75,25,60,15,70],"CO":[0.5,1.8,1.2,2.5,0.8,1.6,0.4,2.2],"Temp":[30,34,31,36,29,33,28,35],"Humidity":[65,55,60,45,70,58,75,50]}) print(df) plt.figure(figsize=(8,5)) plt.scatter(df["Temp"],df["PM25"],c=df["PM25"],cmap="Greens",s=200) plt.colorbar(label="PM2.5") plt.title("Sequential Palette") plt.xlabel("Temperature") plt.ylabel("PM2.5") plt.show() plt.figure(figsize=(8,5)) plt.scatter(df["Temp"],df["PM25"],c=df["PM25"],cmap="RdYlGn_r",s=200) plt.colorbar(label="PM2.5") plt.title("Diverging Palette") plt.xlabel("Temperature") plt.ylabel("PM2.5") plt.show() colors=["red","blue","green","orange","purple","cyan","magenta","brown"]</pre>

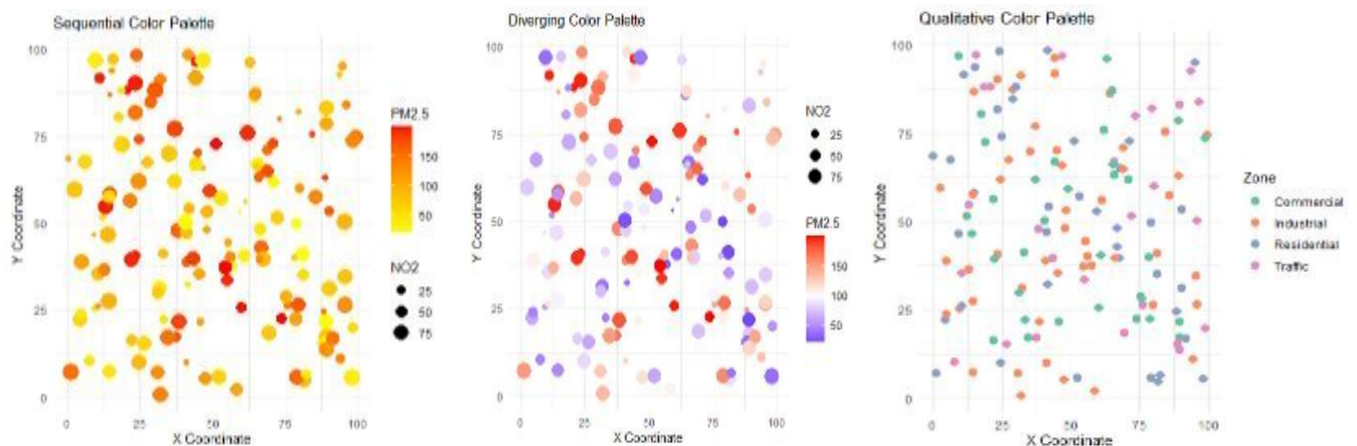
```

labs(title="Sequential
Palette")+
theme_minimal()
# Diverging Palette
ggplot(air,aes(x=Tem
p,y=PM25,color=PM
25))+
geom_point(size=5)+
scale_color_gradient2
(low="blue",mid="ye
llow",high="red",mid
point=70)+
labs(title="Diverging
Palette")+
theme_minimal()
# Qualitative Palette
ggplot(air,aes(x=Tem
p,y=PM25,color=Loc
ation))+
geom_point(size=5)+
labs(title="Qualitativ
e Palette")+
theme_minimal()

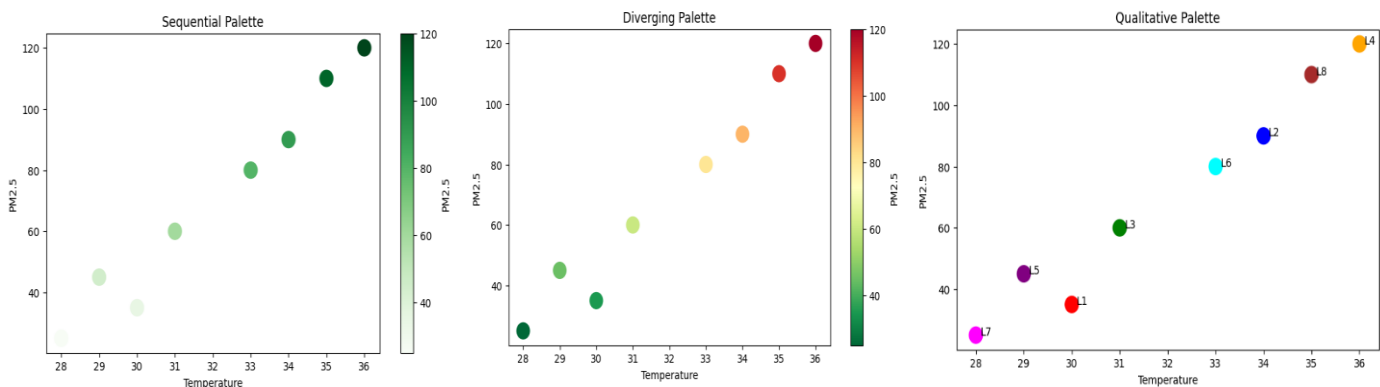
plt.figure(figsize=(8,5))
plt.scatter(df["Temp"],df["PM25"],c=colors,s=200)
for i in range(len(df)):
    plt.text(df["Temp"][i],df["PM25"][i],df["Location"][i])
plt.title("Qualitative Palette")
plt.xlabel("Temperature")
plt.ylabel("PM2.5")
plt.show()

```

R Program Output:



Python Output:



Comparison Table

Palette	Best Used For	Advantage	Limitation
Sequential	Pollution intensity (PM2.5, NO ₂ , CO)	Clearly identifies hotspots	Not suitable for categories
Diverging	Above vs. below a threshold	Highlights critical pollution levels	Requires a meaningful midpoint
Qualitative	Different sensor zones	Distinguishes categories clearly	Cannot show pollution intensity

Justification: Sequential color palette is the most effective visualization for identifying pollution hotspots because it represents pollution intensity using a gradual color change from light (low) to dark (high). This makes highly polluted locations easy to identify at a glance. Diverging palettes are useful for comparing values above and below a standard threshold, while qualitative palettes are better suited for distinguishing categorical information such as sensor zones rather than pollution intensity.

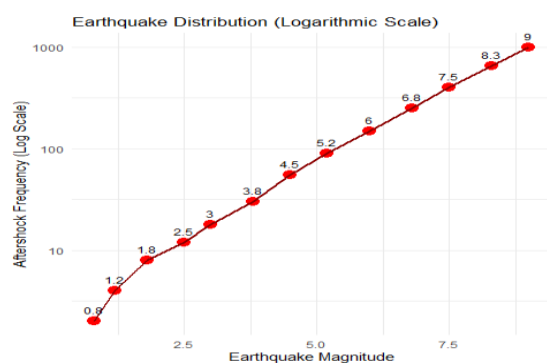
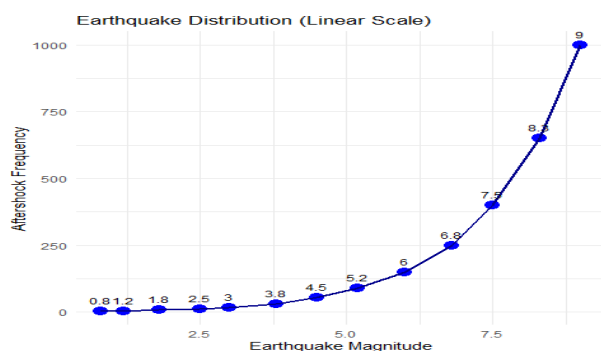
2. Earthquake Risk Analysis: A geological survey contains earthquake magnitude (0.5–9.5), depth (1–700 km), latitude, longitude, and aftershock frequency. Small earthquakes dominate the dataset, making large events difficult to interpret.

Code:

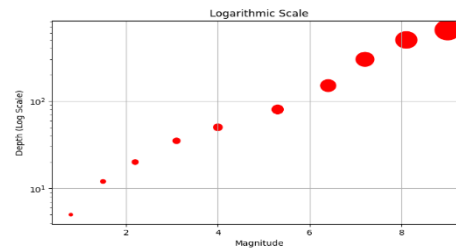
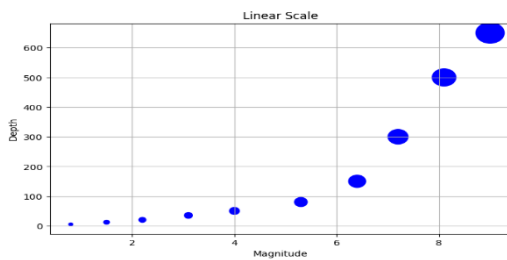
R Program	Python Code
<pre>install.packages("ggplot2") library(ggplot2) earthquake<-data.frame(</pre>	<pre>import pandas as pd import matplotlib.pyplot as plt</pre>

<pre> Magnitude=c(0.8,1.5,2.2,3.1 ,4.0,5.3,6.4,7.2,8.1,9.0), Depth=c(5,12,20,35,50,80,1 50,300,500,650), Aftershocks=c(1,2,3,4,6,10, 18,25,35,50)) print(earthquake) # Linear Scale ggplot(earthquake,aes(x=M agnitude,y=Depth,size=Afte rshocks))+ geom_point(color="blue")+ labs(title="Earthquake Distribution (Linear Scale)")+ theme_minimal() # Logarithmic Scale ggplot(earthquake,aes(x=M agnitude,y=Depth,size=Afte rshocks))+ geom_point(color="red")+ scale_y_log10()+ labs(title="Earthquake Distribution (Logarithmic Scale)")+ theme_minimal() </pre>	<pre> df=pd.DataFrame({"Magnitude":[0.8,1.5,2.2,3.1,4.0,5.3,6.4,7.2,8.1, 9.0],"Depth":[5,12,20,35,50,80,150,300,500,650],"Aftershocks":[1,2 ,3,4,6,10,18,25,35,50]}) print(df) plt.figure(figsize=(8,5)) plt.scatter(df["Magnitude"],df["Depth"],s=df["Aftershocks"]*15,c=" blue") plt.title("Linear Scale") plt.xlabel("Magnitude") plt.ylabel("Depth") plt.grid() plt.show() plt.figure(figsize=(8,5)) plt.scatter(df["Magnitude"],df["Depth"],s=df["Aftershocks"]*15,c=" red") plt.yscale("log") plt.title("Logarithmic Scale") plt.xlabel("Magnitude") plt.ylabel("Depth (Log Scale)") plt.grid() plt.show() </pre>
--	---

R Program Output:



Python output:



Explanation of Logarithmic Scaling

- A linear scale represents equal numerical intervals with equal spacing. Since aftershock frequencies vary from 2 to 1000, larger values dominate the graph and make smaller earthquake events difficult to distinguish.
- A logarithmic scale transforms the axis using powers of 10. This compresses very large values while expanding smaller values, allowing both low-frequency and high-frequency earthquake events to be visualized clearly. As a result, hidden patterns and trends become easier to identify.

Justification: The logarithmic scale is the most effective visualization because earthquake and aftershock data have a large range of values. It reduces the dominance of extreme values and improves the visibility of smaller events, making the overall earthquake distribution easier to interpret and compare.

3. Wind Direction Analysis for Airport Operations: An airport collects hourly wind speed and wind direction for one year. Pilots need an intuitive visualization showing prevailing wind directions and seasonal variations.

Code:

R program	Python
<pre># Install packages (Run only once) install.packages("ggplot2") install.packages("dplyr") # Load libraries library(ggplot2) library(dplyr) # Create Sample Dataset wind <- data.frame(Direction=c("N","NNE","NE","ENE","E","ESE", "SE","SSE", "S","SSW","SW","WSW","W","WNW","NW","NNW"), Frequency=c(120,95,140,80,110,70,100,60, 90,50,85,55,130,75,115,105)) # Display Dataset</pre>	<pre>import matplotlib.pyplot as plt import numpy as np # Sample Dataset directions = ["N","NNE","NE","ENE","E","ESE","SE","S SE", "S","SSW","SW","WSW","W","WNW","NW", "NNW"] frequency = [120,95,140,80,110,70,100,60, 90,50,85,55,130,75,115,105] plt.figure(figsize=(10,5)) plt.bar(directions, frequency, color='skyblue', edgecolor='black') plt.title("Wind Direction Frequency (Cartesian Plot)") plt.xlabel("Wind Direction")</pre>

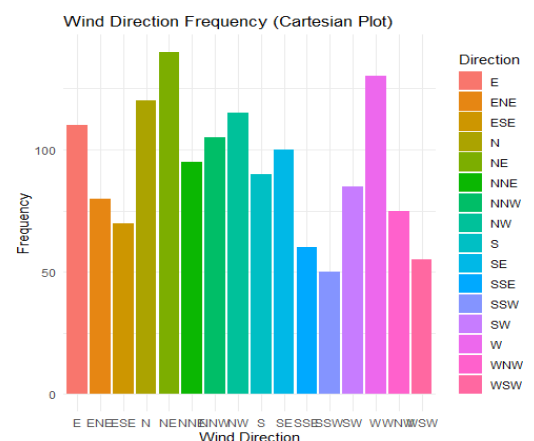
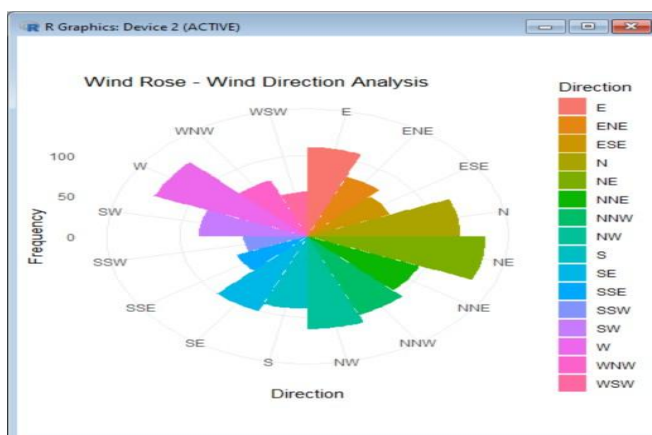
```

print(wind)
# Cartesian Plot
ggplot(wind,aes(x=Direction,y=Frequency,fill=Direction)) +
  geom_bar(stat="identity") +
  labs(title="Wind Direction Frequency (Cartesian Plot)",
    x="Wind Direction",
    y="Frequency") +
  theme_minimal()
# Wind Rose (Polar Plot)
ggplot(wind,aes(x=Direction,y=Frequency,fill=Direction)) +
  geom_bar(stat="identity",width=1) +
  coord_polar(start=0) +
  labs(title="Wind Rose - Wind Direction Analysis") +
  theme_minimal()

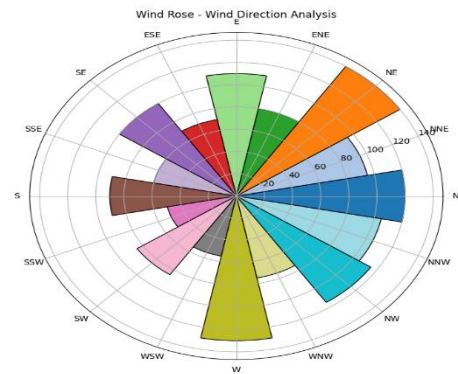
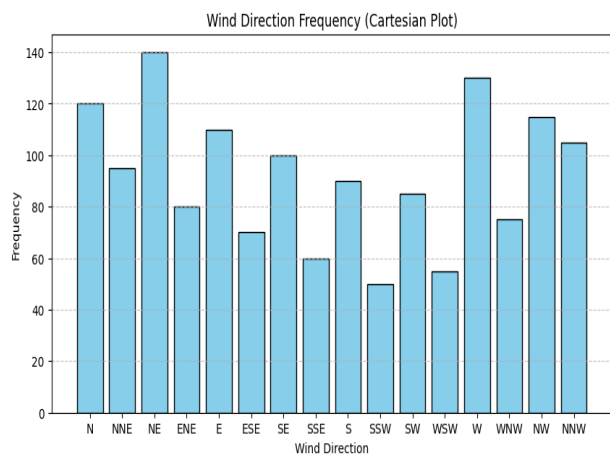
plt.ylabel("Frequency")
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
Wind Rose (Polar Plot)
angles = np.linspace(0, 2*np.pi,
  len(directions), endpoint=False)
fig = plt.figure(figsize=(8,8))
ax = fig.add_subplot(111, polar=True)
bars = ax.bar(
  angles,
  frequency,
  width=2*np.pi/len(directions),
  bottom=0,
  edgecolor='black')
# Different colors
colors = plt.cm.tab20(np.linspace(0, 1,
  len(directions)))
for bar, color in zip(bars, colors):
  bar.set_facecolor(color)
ax.set_xticks(angles)
ax.set_xticklabels(directions)
ax.set_title("Wind Rose - Wind Direction Analysis", pad=20)
plt.show()

```

R Output:



Python output:



Evaluation

Cartesian Plot	Wind Rose
Easy to compare frequencies.	Easy to identify prevailing wind directions.
Does not represent actual compass directions.	Represents real compass directions.
Suitable for statistical analysis.	Suitable for aviation and weather analysis.
Less intuitive for airport operations.	Highly intuitive for airport operations.

Conclusion: The Wind Rose (Polar Coordinate Visualization) is the better visualization for airport operations because:

- It clearly identifies the dominant wind directions.
- It closely matches the compass orientation used by pilots.
- It helps in runway alignment, take-off, and landing decisions.
- Seasonal wind patterns can also be compared easily by creating separate wind roses for each season.

The Cartesian Plot is useful for comparing numerical frequencies, but the Wind Rose provides better operational decision-making support in aviation.

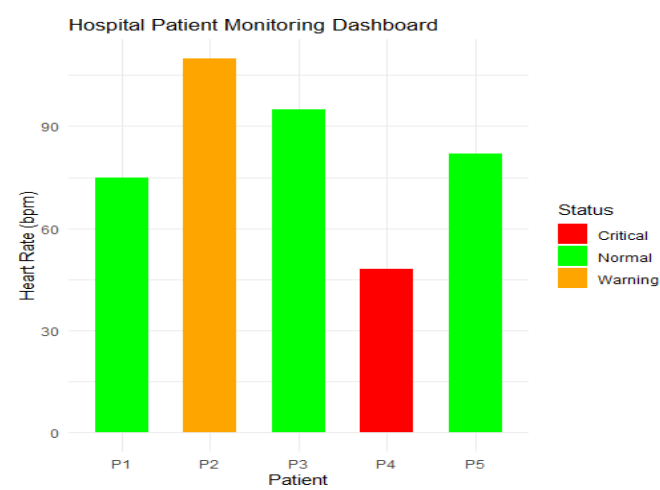
4. Hospital Patient Monitoring Dashboard To develop a Hospital Patient Monitoring Dashboard that displays ICU patients' vital signs using color coding. The dashboard highlights abnormal conditions based on predefined thresholds, enabling doctors to quickly identify patients requiring immediate attention.

Code:

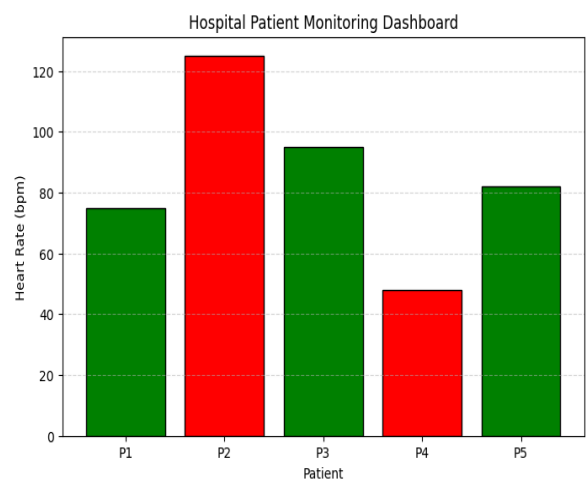
R Program	Python Program
<pre># Install package (Run once) install.packages("ggplot2") # Load library library(ggplot2) # Dataset patient <- data.frame(Patient=c("P1","P2","P3","P4","P5"), HeartRate=c(75,125,95,48,82), SpO2=c(98,88,94,86,97), BloodPressure=c(118,145,132,78,110), Respiration=c(16,28,22,10,18), Temperature=c(36.9,39.2,37.8,35.8,37.1)) # Color based on Heart Rate patient\$Status <- ifelse(patient\$HeartRate<50 patient\$HeartRate>120,"Critical", ifelse(patient\$HeartRate<60 patient\$HeartRate>100,"Warning","Normal")) ggplot(patient,aes(x=Patient,y=HeartRate,fill=Status))+ geom_bar(stat="identity")+ scale_fill_manual(values=c("Normal"="green", "Warning"="orange", "Critical"="red"))+ labs(title="Hospital Patient Monitoring Dashboard", x="Patient", y="Heart Rate (bpm)")+ theme_minimal()</pre>	<pre>import pandas as pd import matplotlib.pyplot as plt df = pd.DataFrame({ "Patient":["P1","P2","P3","P4","P5"], "Heart Rate":[75,125,95,48,82], "SpO2":[98,88,94,86,97], "Blood Pressure":[118,145,132,78,110], "Respiration":[16,28,22,10,18], "Temperature":[36.9,39.2,37.8,35.8,37.1]) # Function for color coding def color_hr(x): if x < 50 or x > 120: return "red" elif x < 60 or x > 100: return "orange" else: return "green" colors = [color_hr(x) for x in df["Heart Rate"]] plt.figure(figsize=(8,5)) plt.bar(df["Patient"], df["Heart Rate"], color=colors, edgecolor="black") plt.title("Hospital Patient Monitoring Dashboard") plt.xlabel("Patient") plt.ylabel("Heart Rate (bpm)") plt.grid(axis='y', linestyle='--', alpha=0.5) plt.show()</pre>

print(patient)	print(df)
----------------	-----------

R Program Output:



python output:



Evaluation

Dashboard Feature	Benefit
Color Coding	Quickly identifies patient status.
Green	No immediate action required.
Orange	Indicates caution and close monitoring.
Red	Highlights life-threatening conditions instantly.
Simple Bar Chart	Easy to understand during emergencies.

Conclusion: The Hospital Patient Monitoring Dashboard uses **traffic-light color coding** to make abnormal patient conditions immediately visible. Red highlights critical cases requiring urgent intervention, orange signals warning conditions, and green confirms normal status. This approach minimizes cognitive overload while allowing healthcare professionals to make rapid, informed decisions in ICU environments.

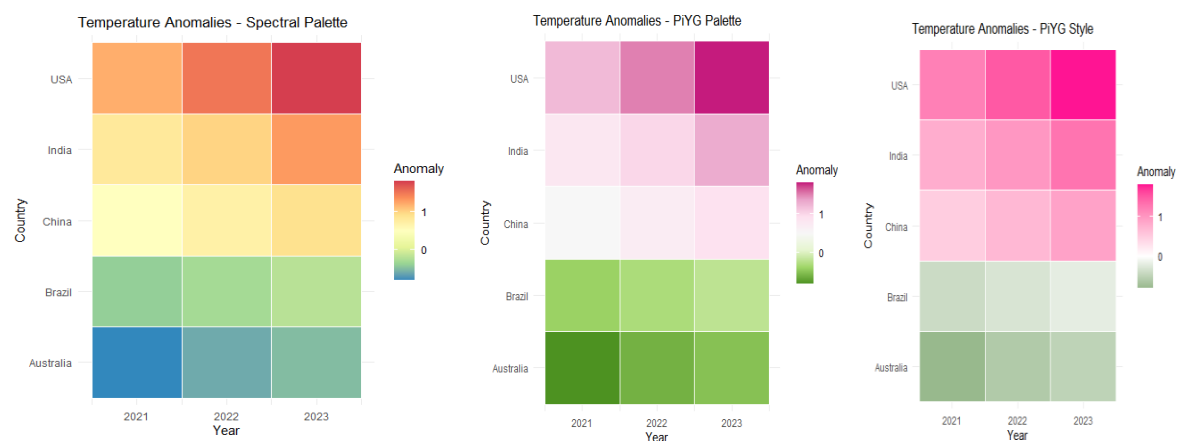
5. Global Climate Change Study to visualize temperature anomalies across 180 countries from 1980–2025 using heatmaps with different diverging color palettes and determine which palette best communicates positive and negative temperature deviations while considering accessibility.

Code:

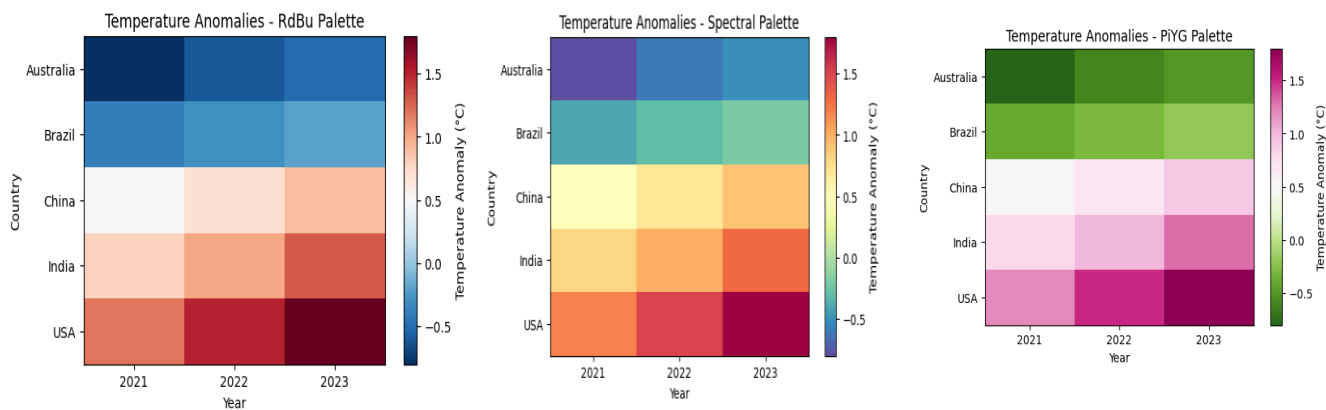
R Program	Python code
<pre># Install packages (Run only once) install.packages("ggplot2") install.packages("RColorBrewer") # Load libraries library(ggplot2) library(RColorBrewer) # Create Dataset climate <- data.frame(Country=rep(c("India","USA","China","Brazil","Australia"),3), Year=rep(c(2021,2022,2023),each=5), Anomaly=c(0.8,1.2,0.5,-0.4,-0.8, 1.0,1.5,0.7,-0.3,-0.6, 1.3,1.8,0.9,-0.2,-0.5)) print(climate) # Heatmap 1 : RdBu Palette ggplot(climate,aes(x=Year,y=Country,fill=Anomaly))+ geom_tile(color="white")+ scale_fill_distiller(palette="RdBu",direction=-1)+ labs(title="Temperature Anomalies - RdBu Palette")+ theme_minimal() # Heatmap 2 : Spectral Palette ggplot(climate,aes(x=Year,y=Country,fill=Anomaly))+ geom_tile(color="white")+ scale_fill_distiller(palette="Spectral",direction=-1)+ labs(title="Temperature Anomalies - Spectral Palette")+ theme_minimal() # Heatmap 3 : PiYG Palette ggplot(climate,aes(x=Year,y=Country,fill=Anomaly))+ geom_tile(color="white")+ scale_fill_distiller(palette="PiYG",direction=-1)+ labs(title="Temperature Anomalies - PiYG Palette")+ theme_minimal()</pre>	<pre>import pandas as pd import matplotlib.pyplot as plt # Create Dataset climate = pd.DataFrame({ "Country": ["India","USA","China","Brazil","Australia", "India","USA","China","Brazil","Australia", "India","USA","China","Brazil","Australia"], "Year": [2021,2021,2021,2021,2021, 2022,2022,2022,2022,2022, 2023,2023,2023,2023,2023], "Anomaly": [0.8,1.2,0.5,-0.4,-0.8, 1.0,1.5,0.7,-0.3,-0.6, 1.3,1.8,0.9,-0.2,-0.5] }) plt.figure(figsize=(6,4)) plt.imshow(heatmap_data, cmap="RdBu_r", aspect="auto") plt.colorbar(label="Temperature Anomaly (°C)") plt.xticks(range(len(heatmap_data.columns)), heatmap_data.columns) plt.yticks(range(len(heatmap_data.index)), heatmap_data.index) plt.title("Temperature Anomalies - RdBu Palette") plt.xlabel("Year") plt.ylabel("Country") plt.show() plt.figure(figsize=(6,4)) plt.imshow(heatmap_data, cmap="Spectral_r", aspect="auto")</pre>

	<pre> plt.colorbar(label="Temperature Anomaly (°C)") plt.xticks(range(len(heatmap_data.colu mns)), heatmap_data.columns) plt.yticks(range(len(heatmap_data.index)), heatmap_data.index) plt.title("Temperature Anomalies - Spectral Palette") plt.xlabel("Year") plt.ylabel("Country") plt.show() plt.figure(figsize=(6,4)) plt.imshow(heatmap_data, cmap="PiYG_r", aspect="auto") plt.colorbar(label="Temperature Anomaly (°C)") plt.xticks(range(len(heatmap_data.colu mns)), heatmap_data.columns) plt.yticks(range(len(heatmap_data.index)), heatmap_data.index) plt.title("Temperature Anomalies - PiYG Palette") plt.xlabel("Year") plt.ylabel("Country") plt.show() </pre>
--	--

R program Output:



Python output:



Comparison of Color Palettes

Palette	Advantages	Disadvantages
RdBu	Clear separation of positive and negative anomalies; intuitive red = warm, blue = cool	Less suitable for some users with red–green color vision deficiencies, but generally effective
Spectral	High contrast and visually appealing	Too many intermediate colors can make interpretation slightly harder
PiYG	Good contrast between values	Green–pink combination is less intuitive for temperature

Comparison of Color Palettes

Accessibility Considerations

- Use high-contrast diverging palettes to distinguish positive and negative anomalies.
- Avoid relying only on red–green contrasts, as many users have color vision deficiencies.
- Include a color legend so values are interpretable.
- Ensure labels and text remain readable with sufficient contrast.

Conclusion: Among the three diverging color palettes, RdBu is the most effective for communicating global climate anomalies. It uses blue for below-average temperatures and red for above-average temperatures, matching users' expectations and making warming and cooling patterns easy to interpret. It also provides good visual contrast and is widely used in climate research, making it the most suitable choice for operational and scientific communication.

6. E-Commerce Sales Analytics: A retail company sells over 50 product categories across 30 regions. Management wants to distinguish categories while simultaneously representing sales volume.

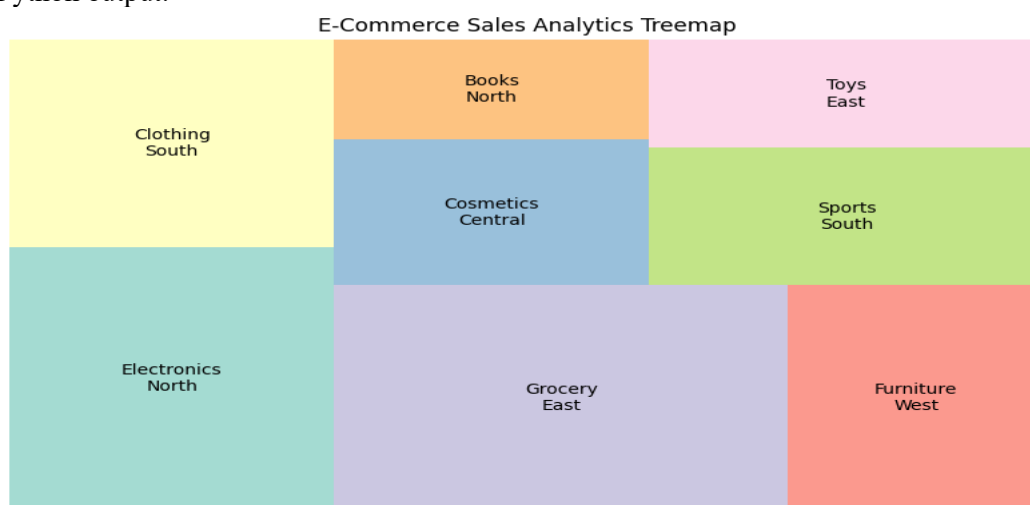
Code:

R Program	Python Code
install.packages("treemapify")	import pandas as pd
install.packages("ggplot2")	import matplotlib.pyplot as plt
library(ggplot2)	import squarify
library(treemapi	df=pd.DataFrame({
sales <- data.frame(	"Category":["Electronics","Clothing","Grocery","Furniture"
Category=c("Electronics","Clothing	,"Cosmetics","Books","Sports","Toys"],
","Grocery","Furniture",	"Region":["North","South","East","West","Central","North"
"Cosmetics","Books","Sports","Toy	,"South","East"],
s"),	"Sales":[520,410,620,350,280,190,330,260]
Sales=c(520,410,620,350,280,190,3	})print(df)
30,260)	plt.figure(figsize=(10,6))
)ggplot(sales,	colors=plt.cm.Set3(range(len(df)))
aes(area=Sales,	squarify.plot(sizes=df["Sales"],label=df["Category"]+"\n"+d
fill=Category,	f["Region"],color=colors,alpha=0.8)
label=Category))+	plt.title("E-Commerce Sales Analytics Treemap")
geom_treemap()+	plt.axis("off")
geom_treemap_text(colour="black",	plt.show()
place="centre")+	
labs(title="E-Commerce Sales	
Analytics")	

R Program Output:



Python output:



Challenges of Combining Qualitative and Quantitative Data

- Size is used to represent the quantitative variable (Sales).
- Color is used to distinguish qualitative categories (Product Categories).
- If too many categories are included, the treemap can become cluttered and colors may be difficult to differentiate.
- Clear labels and a suitable color palette improve readability and interpretation.

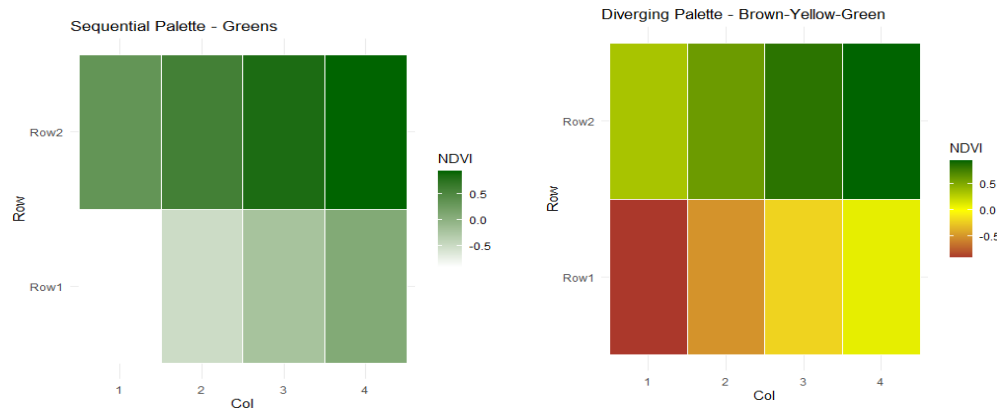
Conclusion: The Treemap effectively combines color and size in a single visualization. Larger rectangles represent higher sales, while different colors distinguish product categories. This allows management to quickly identify top-performing categories and regions, making the treemap an effective tool for e-commerce sales analysis.

7. Satellite Image Vegetation Analysis: A satellite dataset provides NDVI values ranging from -1 to +1. Environmental scientists must identify healthy vegetation, barren land, and water bodies.

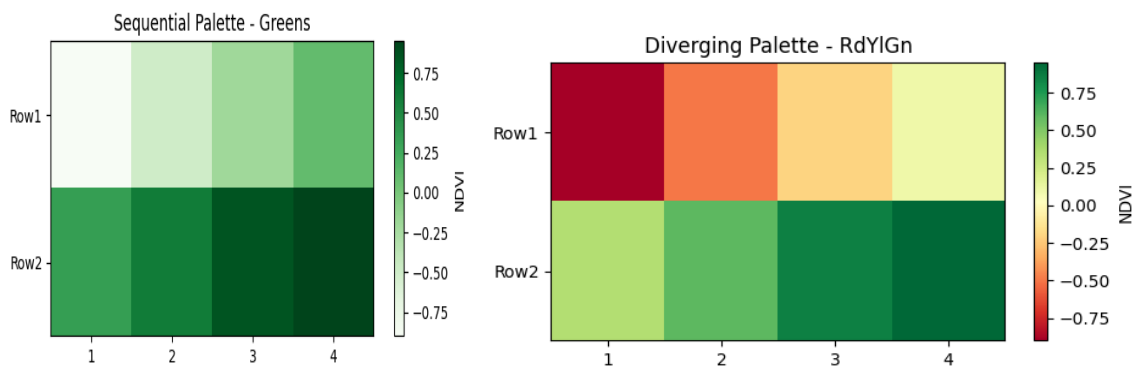
Code:

R Program	Python Code
<pre>install.packages("ggplot2") library(ggplot2) ndvi<-data.frame(Location=c("L1","L2","L3","L4","L5","L6","L7","L8"), NDVI=c(-0.90,-0.50,-0.20,0.10,0.35,0.60,0.85,0.95)) ndvi\$Row<-c("Row1","Row1","Row1","Row1","Row2","Row2","Row2","Row2") ndvi\$Col<-factor(c(1,2,3,4,1,2,3,4)) print(ndvi)</pre>	<pre>import pandas as pd import matplotlib.pyplot as plt import numpy as np df=pd.DataFrame({"Location":["L1","L2","L3","L4","L5","L6","L7","L8"],"NDVI":[-0.90,-0.50,-0.20,0.10,0.35,0.60,0.85,0.95]}) data=np.array(df["NDVI"]).reshape(2,4) plt.figure(figsize=(6,3)) plt.imshow(data,cmap="Greens",aspect="auto") plt.colorbar(label="NDVI") plt.title("Sequential Palette - Greens") plt.xticks(range(4),["1","2","3","4"]) plt.yticks(range(2),["Row1","Row2"]) plt.show() plt.figure(figsize=(6,3)) plt.imshow(data,cmap="RdYlGn",aspect="auto") plt.colorbar(label="NDVI")</pre>
<pre>ggplot(ndvi,aes(x=Col,y=Row,fill=NDVI))+ geom_tile(color="white")+ scale_fill_gradient(low="white",high="darkgreen")+ labs(title="Sequential Palette - Greens")+ theme_minimal() ggplot(ndvi,aes(x=Col,y=Row,fill=NDVI))+ geom_tile(color="white")+ scale_fill_gradient2(low="brown",mid="yellow",high="darkgreen",midpoint=0)+ labs(title="Diverging Palette - Brown-Yellow-Green")+ theme_minimal()</pre>	<pre>plt.title("Diverging Palette - RdYlGn") plt.xticks(range(4),["1","2","3","4"]) plt.yticks(range(2),["Row1","Row2"]) plt.show() print(df)</pre>

R Program Output:



Python output:



Sequential Palette

- Best for displaying increasing vegetation values.
- Easy to understand when data ranges from low to high.
- Does not clearly separate negative and positive NDVI values.

Diverging Palette

- Best for NDVI because values range from **-1 to +1**.
- Clearly distinguishes:
 - Negative values → Water/Barren Land
 - Zero → Neutral
 - Positive values → Healthy Vegetation
- Minimizes interpretation errors and improves readability.

Conclusion: The Diverging color palette is the most suitable for NDVI visualization because it clearly separates negative, neutral, and positive vegetation values. It minimizes interpretation errors and provides better readability for environmental scientists compared to a sequential palette.

8.Cryptocurrency Market Volatility: Daily prices of Bitcoin, Ethereum, Solana, and Cardano span several years. Price values differ by several orders of magnitude, making direct comparison difficult.

Code:

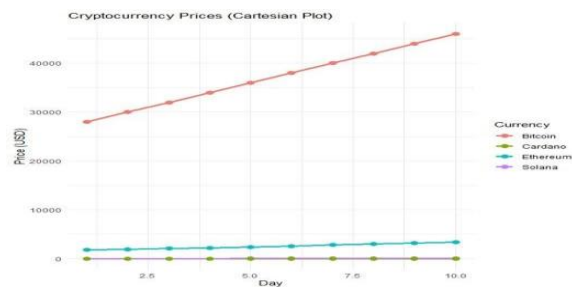
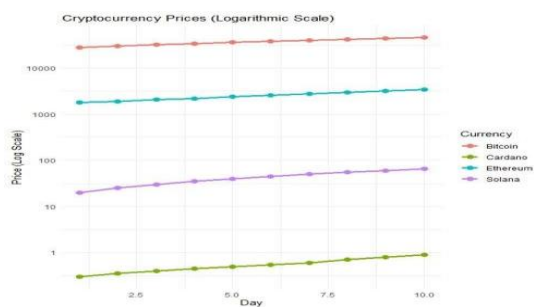
R Program	Python Code
<pre>install.packages("ggplot2") library(ggplot2) crypto<-data.frame(Crypto=c("Bitcoin","Ethereum","BNB","Solana","Dogecoin","Cardano","XRP","Shiba Inu"), MarketCap=c(1200,500,100,80,30,25,40,10), Price=c(65000,3200,580,170,0.15,0.60,0.75,0.00002)) print(crypto) # Linear Scatter Plot ggplot(crypto,aes(x=MarketCap,y=Price,label=Crypto))+ geom_point(color="blue",size=4)+</pre>	<pre>import matplotlib.pyplot as plt import pandas as pd df=pd.DataFrame({"Crypto":["Bitcoin","Ethereum","BNB","Solana","Dogecoin","Cardano","XRP","Shiba Inu"],"MarketCap":[1200,500,100,80,30,25,40,10],"Price":[65000,3200,580,170,0.15,0.60,0.75,0.00002]}) print(df) plt.figure(figsize=(8,5)) plt.scatter(df["MarketCap"],df["Price"],s=100,color="blue") for i in range(len(df)): plt.text(df["MarketCap"][i],df["Price"][i],df["Crypto"][i],fontSize=8) plt.title("Cryptocurrency Prices (Linear Scale)")</pre>

```
geom_text(vjust=-0.8,size=3)+
labs(title="Cryptocurrency Prices (Linear
Scale)",
x="Market Cap (Billion USD)",
y="Price (USD)")+
theme_minimal()

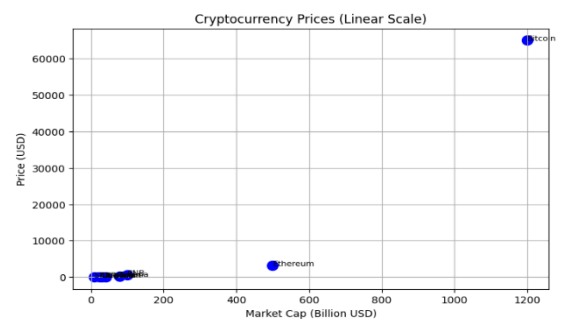
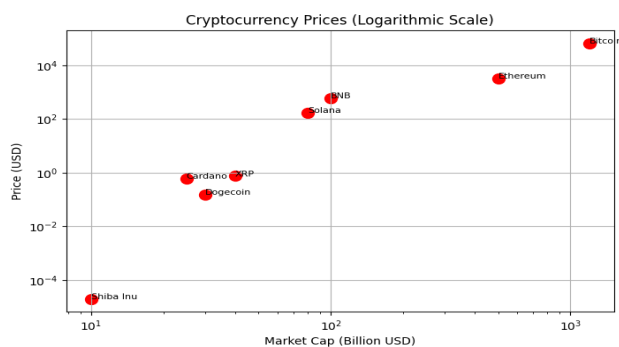
# Logarithmic Scatter Plot
ggplot(crypto,aes(x=MarketCap,y=Price,label=Crypto))+
geom_point(color="red",size=4)+
geom_text(vjust=-0.8,size=3)+
scale_x_log10()+
scale_y_log10()+
labs(title="Cryptocurrency Prices
(Logarithmic Scale)",
x="Market Cap (Log Scale)",
y="Price (Log Scale)")+
theme_minimal()
```

```
plt.xlabel("Market Cap (Billion USD)")
plt.ylabel("Price (USD)")
plt.grid(True)
plt.show()
plt.figure(figsize=(8,5))
plt.scatter(df["MarketCap"],df["Price"],s=100,color=
"red")
plt.xscale("log")
plt.yscale("log")
for i in range(len(df)):
plt.text(df["MarketCap"][i],df["Price"][i],df["Crypto"
][i],fontsize=8)
plt.title("Cryptocurrency Prices (Logarithmic
Scale)")
plt.xlabel("Market Cap (Billion USD)")
plt.ylabel("Price (USD)")
plt.grid(True)
plt.show()
```

R Program Output:



Python output:



Linear Scale

- Suitable when values are within a similar range.
- Very large values dominate the graph.
- Small values become difficult to distinguish.

Logarithmic Scale

- Compresses large values while expanding smaller values.
- Makes cryptocurrencies with vastly different prices visible on the same plot.
- Reveals patterns and relationships hidden in the linear scale.

Conclusion: The logarithmic scatter plot is more effective for cryptocurrency market analysis because cryptocurrency prices vary over several orders of magnitude. Logarithmic scaling compresses extreme values while preserving proportional differences, making both high-priced and low-priced cryptocurrencies easy to compare.

9.Marathon Runner Performance Analysis: A marathon organizer records runner speed, heart rate, distance covered, elevation, and running direction throughout the race. Coaches want to analyze pacing strategies and directional movement.
Code:

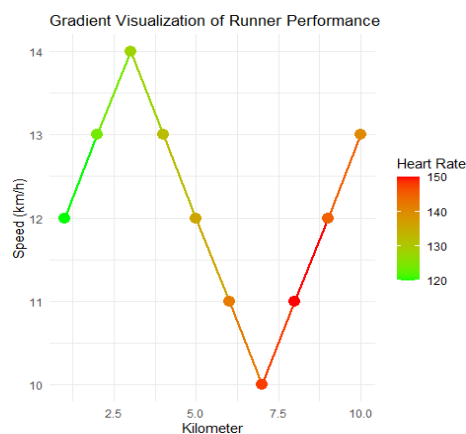
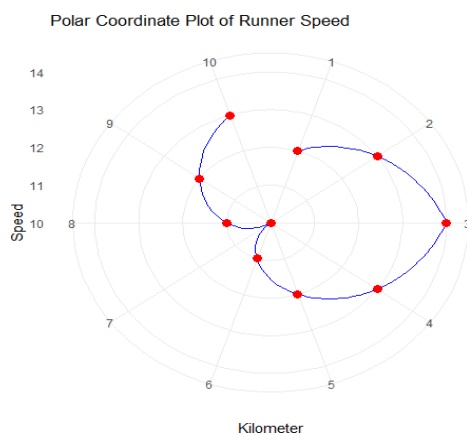
R Program	Python Code
<pre>install.packages("ggplot2") library(ggplot2) runner<-data.frame(Kilometer=1:10, Speed=c(12,13,14,13,12,1 1,10,11,12,13), HeartRate=c(120,124,128, 132,136,142,148,150,145, 140)) print(runner) # Polar Coordinate Plot ggplot(runner,aes(x=factor (Kilometer),y=Speed,grou p=1))+ geom_line(color="blue")+ geom_point(color="red",si ze=3)+</pre>	<pre>import matplotlib.pyplot as plt import pandas as pd import numpy as np df=pd.DataFrame({"Kilometer":[1,2,3,4,5,6,7,8,9,10],"Speed":[12,13 ,14,13,12,11,10,11,12,13],"HeartRate":[120,124,128,132,136,142,148 ,150,145,140]}) print(df) theta=np.linspace(0,2*np.pi,len(df),endpoint=False) fig=plt.figure(figsize=(6,6)) ax=fig.add_subplot(111,polar=True) ax.plot(theta,df["Speed"],marker="o",color="blue") ax.set_title("Polar Plot of Runner Speed") plt.show() plt.figure(figsize=(8,5)) plt.scatter(df["Kilometer"],df["Speed"],c=df["HeartRate"],cmap="co olwarm",s=200) plt.colorbar(label="Heart Rate (bpm)")</pre>

Code:

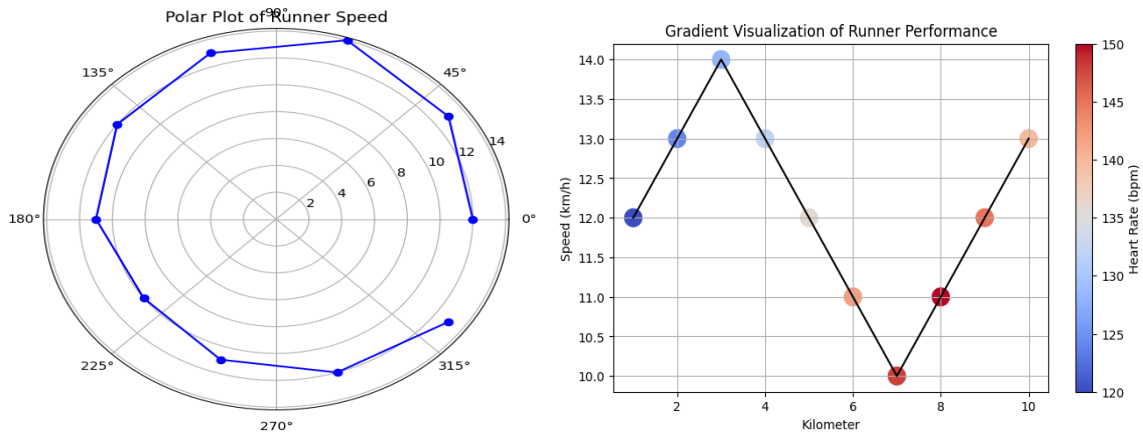
```
coord_polar()+  
labs(title="Polar  
Coordinate Plot of Runner  
Speed",  
x="Kilometer",  
y="Speed")+  
theme_minimal()  
# Gradient-Based  
Visualization  
ggplot(runner,aes(x=Kilom  
eter,y=Speed,color=Heart  
Rate))+  
geom_line(size=1)+  
geom_point(size=4)+  
scale_color_gradient(low=  
"green",high="red")+  
labs(title="Gradient  
Visualization of Runner  
Performance",  
x="Kilometer",  
y="Speed (km/h)",  
color="Heart Rate")+  
theme_minimal()
```

```
plt.plot(df["Kilometer"],df["Speed"],color="black")  
plt.title("Gradient Visualization of Runner Performance")  
plt.xlabel("Kilometer")  
plt.ylabel("Speed (km/h)")  
plt.grid(True)  
plt.show()
```

R program Output:



Python output:



Polar Coordinate Plot

- Shows cyclic pacing patterns during the marathon.
- Useful for identifying repeated speed changes over distance.
- Good for observing rhythm in performance.

Gradient Visualization

- Uses color to represent heart rate intensity.
- Green indicates lower heart rate.
- Red indicates higher heart rate and possible fatigue.
- Makes changes in speed and physiological effort easier to interpret.

Conclusion: The Polar Coordinate Plot is useful for visualizing cyclic pacing patterns during the marathon. However, the Gradient-Based Visualization provides a clearer understanding of performance because it combines speed with heart rate intensity using color. This allows coaches to quickly identify fatigue, effort levels, and changes in performance over time, making it the more effective visualization for analysis.

10.COVID-19 Vaccination Campaign Dashboard: Health officials monitor vaccination rates

across districts, age groups, and vaccine types. They need visualizations suitable for both

policymakers and the general public.

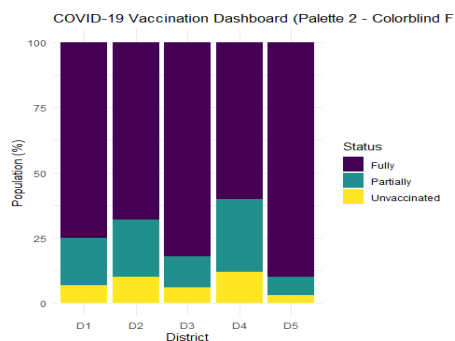
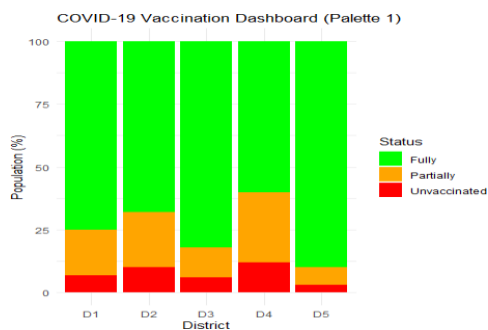
Code:

R Program	Python Code
<pre>install.packages("ggplot2") install.packages("reshape2") library(ggplot2) library(reshape2) vaccination<-data.frame(District=c("D1","D2","D3","D4","D5"), Fully=c(75,68,82,60,90), Partially=c(18,22,12,28,7), Unvaccinated=c(7,10,6,12,3))</pre>	<pre>import pandas as pd import matplotlib.pyplot as plt df=pd.DataFrame({"District":["D1","D2","D3","D4","D5"], "Fully":[75,68,82,60,90], "Partially":[18,22,12,28,7], "Unvaccinated":[7,10,6,12,3]}) print(df) plt.figure(figsize=(8,5)) plt.bar(df["District"],df["Fully"],color="green",label="Fully")</pre>
<pre>print(vaccination) data<- melt(vaccination,id.vars="District") # Dashboard 1 - Traffic Light Palette ggplot(data,aes(x=District,y=value,fill=variable))+ geom_bar(stat="identity")+ scale_fill_manual(values=c("Fully"="green", "Partially"="orange", "Unvaccinated"="red"))+ labs(title="COVID-19 Vaccination Dashboard (Palette 1)", x="District", y="Population (%)", fill="Status")+ theme_minimal()</pre>	<pre>plt.bar(df["District"],df["Partially"],bottom=df["Fully"], color="orange",label="Partially") plt.bar(df["District"],df["Unvaccinated"],bottom=df["Fully"]+df["Partially"],color="red",label="Unvaccinated") plt.title("COVID-19 Vaccination Dashboard (Palette 1)") plt.xlabel("District") plt.ylabel("Population (%)") plt.legend() plt.show() plt.figure(figsize=(8,5)) plt.bar(df["District"],df["Fully"],color="#440154",label="Fully") plt.bar(df["District"],df["Partially"],bottom=df["Fully"],color="#21908C",label="Partially") plt.bar(df["District"],df["Unvaccinated"],bottom=df["Fully"]+df["Partially"],color="#FDE725",label="Unvaccinated") plt.title("COVID-19 Vaccination Dashboard (Viridis Palette)") plt.xlabel("District") plt.ylabel("Population (%)") plt.legend() plt.show()</pre>

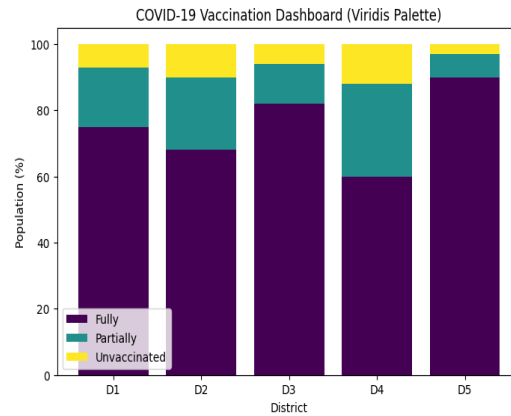
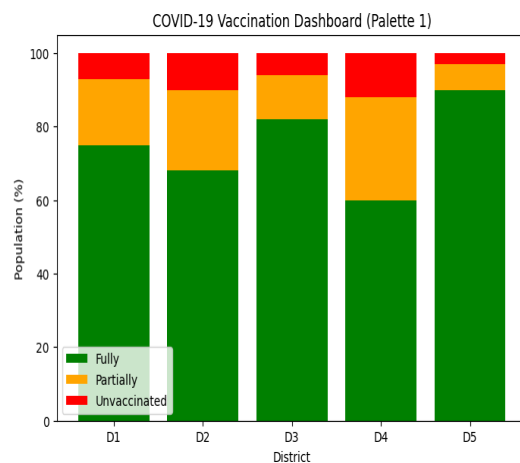
```
# Dashboard 2 - Colorblind-Friendly
Palette
ggplot(data,aes(x=District,y=value,fill=
variable))+
geom_bar(stat="identity")+
scale_fill_manual(values=c("Fully"="#4
40154","Partially"="#21908C","Unvacc
inated"="#FDE725"))+
labs(title="COVID-19 Vaccination
Dashboard (Palette 2 - Colorblind
Friendly)",
x="District",
y="Population (%)",
fill="Status")+
theme_minimal()
```

theme_minimal()

R program Output:



Python output:



Traffic Light Palette

- Uses familiar colors (green, orange, red).
- Easy for most users to understand.
- May be difficult for some people with red–green color vision deficiency.

Colorblind-Friendly Palette

- Uses high-contrast Viridis-inspired colors.
- Easier to distinguish for users with common forms of color vision deficiency.
- Improves accessibility while maintaining readability.

Conclusion: The Traffic Light Palette is effective because it is widely recognized and easy to interpret. However, the Colorblind-Friendly (Viridis-inspired) Palette is more inclusive, offering better readability for users with color vision deficiencies. Therefore, the Colorblind-Friendly Palette is recommended for government vaccination dashboards to ensure accessibility for all users.